

Idoneidad de un motor C++ existente para escalar el modelo de depredación de ovejas

Análisis técnico comparativo

Preparado para: Prof. Mauricio Marín
Manuel Manríquez (autor de `sim2-agricultores`)

Preparado por: Francisco Parra

Fecha: 13 de julio de 2026

Objeto: Contrastar el motor C++ propuesto contra `swarm-abm` como vehículo para escalar el modelo SIGRID (depredación de ganado).

Resumen ejecutivo. Para *escalar* el problema de las ovejas depredadas se requiere calibrar el modelo y hacerle análisis de sensibilidad de forma auditable, y ambas cosas descansan en la **reproducibilidad** (mismo input → mismo output). Una prueba en vivo muestra que el código C++ propuesto, *tal como está hoy*, **no reproduce sus propias corridas**: dos ejecuciones con configuración idéntica dan resultados distintos. El mismo experimento sobre `swarm-abm` es reproducible bit a bit. Esto no significa que el C++ sea “inutilizable”: la falla es arreglable en principio. El punto es de **esfuerzo y riesgo**: usarlo para esta tarea implica reconstruir en C++ el motor espacial, el diseño de experimentos y la garantía de reproducibilidad que `swarm-abm` ya provee, tiene auditado y publicado.

1. Qué es el código C++ propuesto

`sim2-agricultores` (~5.160 líneas de C++) es un simulador de **mercados agrícolas** —agricultores, feriantes, consumidores, mercado mayorista, terrenos y productos, con eventos de sequía, helada y ola de calor— y es una pieza sólida en su dominio. Su arquitectura es de **simulación por eventos discretos** (DES): una *future event list*, manejadores de eventos y colas de mensajes.

El modelo de ovejas (SIGRID) es de naturaleza distinta: **espacial y por pasos** (grilla, dinámica depredador–presa evaluada por *tick*). De ahí dos observaciones de fondo, previas a la reproducibilidad:

- El código C++ **no contiene primitivas espaciales** (grilla, vecindades). Llevar SIGRID ahí obliga a construir esa infraestructura desde cero.
- El **paradigma no calza**: un DES orientado a transacciones de mercado no expresa naturalmente la dinámica espacial sincrónica de depredación.

2. Prueba en vivo: el C++ propuesto no es reproducible

Se compiló y ejecutó el simulador C++ **dos veces con configuración idéntica** (`sim_config.json` sin cambios), en un contenedor autocontenido (PostgreSQL 16 + g++ 13 + libpqxx). Se comparó el vector completo de resultados agregados mediante hash MD5:

```

Corrida 1 - exit 0 - 1260 filas - hash 089cdd94c7695f77...
Corrida 2 - exit 0 - 1260 filas - hash 9f344b0f1e960c3d... ← distinto
65 de 1260 filas (misma clave) tienen VALOR distinto entre las dos corridas.
Ej.: “COMPRA DE FERIAANTE A AGRICULTOR”, t=16, prod=18 → 36.360 vs 28.620
(−21 %)
VEREDICTO: NO REPRODUCIBLE.

```

Causa. Cada fuente de aleatoriedad del código se siembra con `std::mt19937(std::random_device{}())` —desde la entropía del sistema operativo, sin semilla fija en ninguna parte del API (aparece en al menos 8 archivos)—. Con el código actual no hay forma de forzar dos corridas idénticas.

Matiz honesto. Para estadísticas agregadas de mercado por Monte Carlo (correr N veces y promediar), esta decisión es legítima. Pero es una limitación para calibración y análisis de sensibilidad —justamente lo que “escalar el problema” requiere— y para depurar (un resultado anómalo no se puede reproducir).

3. Prueba en vivo: swarm-abm sí es reproducible

El mismo experimento sobre el modelo de ovejas real en `swarm-abm` (dos corridas con la misma semilla):

```

Corrida 1 - hash de la salida: faa5b35e733da2cb...
Corrida 2 - hash de la salida: faa5b35e733da2cb... ← idéntico
VEREDICTO: REPRODUCIBLE BIT A BIT.

```

No es accidente de una corrida: el determinismo de `swarm-abm` está garantizado por construcción (RNG ChaCha8 sembrable, semilla por-agente, ejecución paralela bit-idéntica a la secuencial, identidad entre x86-64 y WebAssembly) y **verificado en integración continua** en cada cambio.

4. Matriz de idoneidad para escalar el modelo de ovejas

Lo que “escalar” exige	sim2-agricultores	swarm-abm
Reproducibilidad bit a bit	no (demostrado)	sí (demostrado + CI)
Ya implementa el modelo de ovejas	no (es mercados)	sí (SIGRID vs Mesa)
Primitivas espaciales (grilla, vecindades)	no (DES no espacial)	sí (tres espacios)
Análisis de sensibilidad global nativo	no (a construir)	sí (Sobol/Morris/LHS)
Calibración	no (manual)	sí (DE, $\sim 400\times$ vs Python)
Determinismo bajo paralelismo	no	sí (paralelo = secuencial)
Dependencias para correr	PostgreSQL	ninguna (+ WASM)
Estado del código	tesis, <code>-fpermissive</code>	crates.io, 3 auditorías, CI

5. Sobre el rendimiento (honesto)

No es posible comparar la velocidad de ambos directamente: son modelos y paradigmas distintos, y cronometrar uno contra otro no mediría nada. Lo que sí está medido con rigor es que `swarm-abm` *no sacrifica rendimiento por ser reproducible*: es 45–184× más rápido que Mesa (Python) y está a solo $\sim 1,5$ – $2,6\times$ del motor de ABM nativo más rápido que existe (krABMaga, Rust). Es decir, el argumento “hay que ir a C++ por velocidad” no se sostiene: un motor en Rust

ya está en la banda de rendimiento nativo *y además* es reproducible y trae análisis de sensibilidad integrado.

6. Avance del trabajo: port C++ del modelo, validado contra swarm-abm

Adoptado el esquema de dos motores —swarm-abm como referencia reproducible y oráculo de validación, C++ como motor de producción/escala— el trabajo ya arrancó y muestra resultados concretos.

Benchmark de escala (¿hace falta distribución?)

Un kernel depredador–presa espacial *idéntico* en C++ (escrito a mano, con índice de celdas) y en swarm-abm (su motor), escalando agentes en un solo nodo. Ambos escalan **linealmente** (mismo orden algorítmico); el C++ a mano es **~1,9× más rápido** —un factor constante, no creciente—. A 40.000 agentes ambos corren un paso en milisegundos: la escala que se discute cabe holgadamente en **un solo nodo**. El motor en Rust está en la banda de rendimiento nativo, y a cambio del $\sim 2\times$ entrega reproducibilidad y análisis de sensibilidad integrado.

Port del modelo de ovejas, verificado por el oráculo

Se implementó en C++ el modelo de ovejas (subconjunto de *screening*: oveja, zorro, perro guardián, liebre, chilla), **reproducible** (sembrado, a diferencia de sim2-agricultores), y validado **distribucionalmente contra swarm-abm** —la misma metodología que la paridad vs Mesa—:

Etapa	Especies	Paridad (Pearson r)
1	oveja + zorro	0,9986
2	+ perro guardián	0,9949
3	+ liebre + chilla	0,9997

El oráculo cumplió su función de verificación: (i) **cazó un bug** —el CLI de swarm-abm documentaba un parámetro (`--fox-density`) que no aplicaba—; y (ii) mostró que la disuasión de perros es un **sistema de alta varianza** que exige más réplicas para una comparación estable. El próximo paso es paralelizar el C++ con **OpenMP**, con swarm-abm verificando que la paralelización no rompe los resultados —el punto donde el motor de referencia se vuelve indispensable, porque los errores de concurrencia son silenciosos.

7. Conclusión

No se afirma que el código C++ sea inutilizable —sería impreciso—. Hay dos cosas distintas: (i) la no-reproducibilidad es un defecto **arreglable en principio**, cableando una semilla, aunque hacerlo bien (y mantener determinismo si se paraleliza) es trabajo de ingeniería no trivial; y (ii) el obstáculo mayor es que el código **no es el modelo ni el paradigma**: usarlo para las ovejas implica reimplementar el modelo espacial depredador–presa en un framework de mercados que no lo soporta.

El encuadre correcto no es “se puede o no se puede”, sino **esfuerzo y riesgo frente a reutilizar lo ya hecho**. Usar este C++ para escalar el problema significa reconstruir —con menos garantías y semanas de trabajo— el motor espacial, el diseño de experimentos y la reproducibilidad que **swarm-abm** ya provee, tiene auditado y publicado. El modelo de ovejas ya corre ahí, validado contra Mesa y reproducible bit a bit, y fue con su análisis de sensibilidad nativo que se identificó al número de perros como la principal palanca de manejo.

La comparación es reproducible: el arnés de la prueba (Dockerfile + script) está disponible y repite ambos lados en un comando. swarm-abm: <https://github.com/franciscoparrao/swarm-abm> / <https://crates.io/crates/swarm-abm>.